

```
#include<mpi.h>

#include<stdio.h>

int main(int argc, char *argv[]) {

    int rank, size;

    int number;

    // Initialize the MPI environment
    MPI_Init(&argc, &argv);

    // Get the rank of the process
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    // Get the total number of processes
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (size < 2) {
        if (rank == 0) {
            printf("This program requires at least two processes.\n");
        }
        MPI_Finalize();
        return 0;
    }

    if (rank == 0) {
        // Process 0 sends a number to process 1
        number = 123;

        MPI_Send(&number, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);

        printf("Process 0 sent number %d to process 1\n", number);
    }

    else if (rank == 1) {
```

```
// Process 1 receives the number from process 0

MPI_Recv(&number, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);

printf("Process 1 received number %d from process 0\n", number);
}


// Finalize the MPI environment

MPI_Finalize();

return 0;
}
```